

FREE SAMPLE - MODULE 1 + CHEAT SHEET

INTERVIEW PREP PACK

Playwright Interview Mastery

80 fully answered Playwright + TypeScript interview questions, 400+ practice questions, cheat sheet, mock test and sample projects

8

MODULES

80

ANSWERED QS

486

PRACTICE QS

20

MOCK TEST QS

2

SAMPLE PROJECTS

By Ravi

Edition 1.0 - October 2026

Playwright 1.59 | TypeScript | @playwright/test

Not affiliated with Microsoft or the Playwright project. Playwright is an open-source project by Microsoft. Questions are original and reflect topics commonly tested in QA/SDet interviews at service and product companies; they are not leaked or official questions from any company. No job or interview outcome is guaranteed.
(c) 2026 Ravi. Free sample - you are welcome to share this PDF unmodified.

Contents

How to use this pack	2
Module 1: Playwright Basics & Architecture	3
Q1.01 What is Playwright and how does it differ from Selenium WebDriver?	3
Q1.02 Explain Playwright's architecture: test runner, client library, and how commands reach the browser.	5
Q1.03 Explain the difference between Browser, BrowserContext and Page in Playwright.	6
Q1.04 Which browsers and channels does Playwright support, and how do you run branded Chrome or Edge?	8
Q1.05 How do you set up a new Playwright Test project and what does the generated structure contain?	9
Q1.06 Playwright Library vs Playwright Test: when would you use each?	11
Q1.07 What is codegen and how should it be used in a real project?	12
Q1.08 How do you handle new tabs, popups and multiple windows in Playwright?	14
Q1.09 How do you work with iframes in Playwright?	15
Q1.10 How do you handle JavaScript dialogs, file uploads and downloads in Playwright?	17
Cheat Sheet	19
What is in the full pack	22

START HERE

How to use this pack

1. Study one module per day. Read each question, try answering aloud for 60-90 seconds, then compare with the 10-point answer.
2. Type and run the code snippets instead of only reading them. Most spec snippets are self-contained and run as-is with `npx playwright test`.
3. Use the Common mistakes and Likely follow-ups sections to rehearse the second and third questions an interviewer usually asks.
4. Use the Practice Question Index (Appendix A) for mock rounds with a friend, or record yourself answering five random questions per module.
5. Take the 30-minute timed mock test at the end of the week, then revise the modules where you lost marks.

About the code.

- All snippets target @playwright/test 1.59 with TypeScript. Every TypeScript snippet was type-checked with strict `tsc`, and the self-contained spec snippets were executed in Chromium against Playwright 1.59.1.
- Runnable specs use `page.setContent()`, `page.route()` or the public demo app at <https://demo.playwright.dev/todomvc/> so they need no private backend.
- Config, YAML, shell and Dockerfile snippets are illustrative. Adapt paths, URLs and secrets to your project and keep credentials in environment variables.

#	Module	Questions	Beginner / Intermediate / Advanced
1	Playwright Basics & Architecture	10	5 / 5 / 0
2	Locators Mastery (<i>full pack</i>)	10	4 / 4 / 2
3	Auto-waiting & Assertions (<i>full pack</i>)	10	3 / 4 / 3
4	Test Runner, Fixtures & Configuration (<i>full pack</i>)	10	3 / 5 / 2
5	Authentication & storageState (<i>full pack</i>)	10	2 / 4 / 4
6	Network Mocking & API Testing (<i>full pack</i>)	10	3 / 4 / 3
7	Debugging, Trace Viewer & Flakiness (<i>full pack</i>)	10	4 / 5 / 1
8	CI/CD, Scaling & Framework Design (<i>full pack</i>)	10	2 / 4 / 4

Not affiliated with Microsoft or the Playwright project. Playwright is an open-source project by Microsoft. Questions are original and reflect topics commonly tested in QA/SDET interviews at service and product companies; they are not leaked or official questions from any company. No job or interview outcome is guaranteed.

MODULE 1

Playwright Basics & Architecture

Builds the mental model every Playwright interview starts with: what Playwright is, how a command travels from your test to the browser, and how Browser, BrowserContext and Page fit together. It then covers project setup, Library vs Test, codegen, and the everyday browser surfaces - tabs, popups, iframes, dialogs, uploads and downloads - that interviewers use to separate tool users from engineers who understand the tool.

#	Question	Level
Q1.01	What is Playwright and how does it differ from Selenium WebDriver?	Beginner
Q1.02	Explain Playwright's architecture: test runner, client library, and how commands reach the browser.	Intermediate
Q1.03	Explain the difference between Browser, BrowserContext and Page in Playwright.	Beginner
Q1.04	Which browsers and channels does Playwright support, and how do you run branded Chrome or Edge?	Beginner
Q1.05	How do you set up a new Playwright Test project and what does the generated structure contain?	Beginner
Q1.06	Playwright Library vs Playwright Test: when would you use each?	Intermediate
Q1.07	What is codegen and how should it be used in a real project?	Beginner
Q1.08	How do you handle new tabs, popups and multiple windows in Playwright?	Intermediate
Q1.09	How do you work with iframes in Playwright?	Intermediate
Q1.10	How do you handle JavaScript dialogs, file uploads and downloads in Playwright?	Intermediate

Q1.01 What is Playwright and how does it differ from Selenium WebDriver?

Beginner M1-Q01 | Module 1

ANSWER IN 10 POINTS

1. Playwright is an open-source browser automation framework from Microsoft that drives Chromium, Firefox and WebKit through one API. It ships official bindings for JavaScript/TypeScript, Python, Java and .NET, and for Node it includes its own test runner, `@playwright/test`.
2. Selenium WebDriver implements the W3C WebDriver standard: every command is an HTTP request from the client library to a per-browser driver such as chromedriver or geckodriver, which then talks to the browser. Playwright instead keeps one persistent connection open and speaks each browser's automation protocol directly.
3. Because the connection is two-way, Playwright receives browser events (network requests, console messages, dialogs, new pages) as they happen. That enables built-in auto-waiting, network interception with `page.route()`, and event APIs like `page.waitForEvent('popup')` without extra plugins.
4. Auto-waiting is the biggest day-to-day difference. Before an action, Playwright checks that the element is attached, visible, stable, enabled and able to receive events, so explicit waits and `Thread.sleep`-style pauses are rarely needed.
5. Assertions are web-first: `await expect(locator).toHaveText('Saved')` keeps retrying until the condition holds or the timeout expires. In Selenium you typically build this yourself with `WebDriverWait` and `expected`

conditions.

6. Isolation is cheap. Each test gets a fresh **BrowserContext** (an incognito-like profile) in milliseconds, whereas a typical Selenium suite starts a new browser session per test or risks leaking cookies between tests.
7. Playwright Test bundles what a Selenium stack usually assembles from separate libraries: parallel workers, fixtures, retries, HTML reporter, screenshots, video and the Trace Viewer for step-by-step debugging of CI failures.
8. Selenium still has real strengths: it drives genuine branded browsers including Safari through safaridriver, has a huge ecosystem, Selenium Grid for distributed runs, and pairs with Appium for native mobile apps. Playwright's WebKit is a patched open-source build, not Safari itself, and mobile support is emulation.
9. Choose based on context: Playwright suits modern web apps and teams that want fast, reliable, low-maintenance UI and API tests; Selenium may remain the pragmatic choice for large existing suites, strict real-Safari requirements or native mobile coverage.
10. Interview-ready summary: Playwright is a modern multi-browser framework with a persistent protocol connection, auto-waiting, web-first assertions, cheap context isolation and a batteries-included runner, while Selenium is the W3C WebDriver standard with broader legacy, real-browser and mobile ecosystem coverage.

CODE EXAMPLE

TypeScript test - executed in Chromium (Playwright 1.59.1)

```
import { test, expect } from '@playwright/test';

test('no explicit waits needed for a typical flow', async ({ page }) => {
  await page.goto('https://demo.playwright.dev/todomvc/');

  const input = page.getByPlaceholder('What needs to be done?');
  await input.fill('Prepare Playwright interview');
  await input.press('Enter');

  // Actions auto-wait for actionability; assertions retry until they pass
  await expect(page.getByTestId('todo-title')).toHaveText(['Prepare Playwright interview']);
  await expect(page.getByTestId('todo-count')).toContainText('1 item left');

  await page.getByTestId('todo-item').getByRole('checkbox').check();
  await expect(page.getByTestId('todo-item')).toHaveClass(/completed/);
});
```

COMMON MISTAKES

- Claiming Playwright is 'Selenium with a new API' - it does not use the WebDriver HTTP protocol or per-browser driver binaries at all.
- Porting Selenium habits such as fixed sleeps, `page.waitForTimeout()` everywhere, or manual polling loops instead of relying on auto-waiting and web-first assertions.
- Stating that Playwright tests real Safari or real mobile devices - WebKit is a Playwright build of the engine and mobile is device emulation.

LIKELY FOLLOW-UPS

- Your team has 2,000 Selenium tests in Java. How would you plan a migration to Playwright, or would you?
- Which actionability checks does Playwright run before a click, and how would you debug a click that times out?

Official docs: playwright.dev/docs/intro | playwright.dev/docs/actionability | playwright.dev/docs/best-practices

Q1.02 Explain Playwright's architecture: test runner, client library, and how commands reach the browser.

Intermediate M1-Q02 | Module 1

ANSWER IN 10 POINTS

1. Think of Playwright in three layers: the test runner (`@playwright/test`) that discovers and schedules tests, the client API you call (`page`, `locator`, `expect`), and the driver/server layer that translates API calls into browser protocol messages.
2. The test runner is a Node.js orchestrator. It reads `playwright.config.ts`, spawns worker processes, and each worker launches its browser once and creates fresh fixtures (`context`, `page`) per test. Workers never share browser state, which is why parallel runs stay isolated.
3. In Node.js the client and the driver run in the same process, so an API call is dispatched in-process. The Python, Java and .NET bindings ship a bundled Node.js driver; the language client starts it as a child process and exchanges messages with it over a pipe.
4. The driver talks to Chromium over the Chrome DevTools Protocol (CDP). Firefox and WebKit are driven through Playwright-patched browser builds that expose Playwright's own automation protocol, which is why Playwright downloads specific browser revisions matching each release.
5. There is one long-lived connection per browser rather than an HTTP request per command as in classic WebDriver. Commands and events flow both ways over it, so the driver knows immediately about navigations, network traffic, dialogs and new pages.
6. Much of the work happens close to the browser: an injected script resolves locators inside the page and actionability checks run there too. A single `locator.click()` therefore becomes one high-level command that waits, scrolls into view and clicks, instead of many client round trips.
7. Remote and debugging variants use the same model: `browserType.connect()` attaches to a remote Playwright server over WebSocket, and `browserType.connectOverCDP()` attaches to an already running Chromium. For Chromium you can even open a raw `CDPSession` with `context.newCDPSession(page)`.
8. Gotcha: protocol-level access is browser specific. CDP sessions work only in Chromium, so code that uses them must skip or branch on `browserName` to keep Firefox and WebKit projects green.
9. Practical debugging of this pipeline uses `DEBUG=pw:api` for verbose API logs, the Trace Viewer for action-by-action DOM snapshots and network calls, and `--debug` to step through commands in the Playwright Inspector.
10. Interview-ready summary: the runner schedules tests across workers, the client API sends high-level commands to a driver (in-process for Node, a bundled Node driver for other languages), and the driver uses CDP for Chromium and Playwright's own protocol for patched Firefox and WebKit over one persistent, event-driven connection.

CODE EXAMPLE

TypeScript test - executed in Chromium (Playwright 1.59.1)

```
import { test, expect } from '@playwright/test';

test('fixtures expose the Browser > Context > Page chain', async ({ browser, context, page, browserName }) => {
  expect(page.context()).toBe(context);
  expect(context.browser()).toBe(browser);
  expect(browser.browserType().name()).toBe(browserName);

  await page.setContent('<button>Pay now</button><p role="status"></p><script>' +
    'document.querySelector("button").addEventListener("click", () => {' +
    '  document.querySelector("p").textContent = "Payment queued"; });</script>');

  // One API call: locator resolution, actionability checks and the click
  // all run browser-side over the persistent connection
  await page.getByRole('button', { name: 'Pay now' }).click();
  await expect(page.getByRole('status')).toHaveText('Payment queued');

  // Raw protocol access exists only for Chromium (CDP)
  if (browserName === 'chromium') {
    const cdp = await context.newCDPSession(page);
    const info = await cdp.send('Browser.getVersion');
    expect(info.product).toContain('Chrome');
  }
});
```

COMMON MISTAKES

- Saying Playwright uses WebDriver or Selenium Grid under the hood, or that every command is a separate HTTP call.
- Claiming all three browsers are driven over CDP - only Chromium is; Firefox and WebKit use patched builds with Playwright's own protocol.
- Using `newCDPSession` in shared helpers without guarding for Chromium, which breaks Firefox and WebKit projects.

LIKELY FOLLOW-UPS

- Why does Playwright need its own patched Firefox and WebKit builds instead of using the versions installed on the machine?
- How would you run Playwright tests against browsers hosted on a remote machine or a cloud grid?

Official docs: playwright.dev/docs/library | playwright.dev/docs/browsers | playwright.dev/docs/test-configuration

Q1.03 Explain the difference between Browser, BrowserContext and Page in Playwright.

Beginner M1-Q03 | Module 1

ANSWER IN 10 POINTS

1. **Browser** is one launched browser process - Chromium, Firefox or WebKit. Launching is the most expensive step, so Playwright Test starts one browser per worker process and reuses it across the tests that worker runs.
2. **BrowserContext** is an isolated, incognito-like session inside that browser. Each context has its own cookies, localStorage, sessionStorage, cache and permissions, so nothing leaks between contexts.
3. **Page** is a single tab (or popup window) that belongs to exactly one context. Pages in the same context share cookies and storage, which is how you test multi-tab journeys for one signed-in user.

4. Creating a context is cheap (milliseconds), which is why Playwright Test gives every test a fresh `context` and `page` fixture - full isolation without the cost of relaunching the browser.
5. Context-level options define the environment: `viewport`, `locale`, `timezoneId`, `geolocation`, `permissions`, `storageState`, `httpCredentials`, `colorScheme`, `userAgent` and more. In Playwright Test you usually set them through `use` in the config.
6. Multi-user scenarios, for example an admin approving a request raised by a customer, are modelled with two contexts from the same `browser` fixture so each user has an independent session inside one test.
7. Hooks exist at both levels: `context.route()` and `context.on('page')` apply to every page in the context, while `page.route()` applies only to that one tab.
8. Closing a context closes all its pages and discards its storage; closing the browser closes every context. Playwright Test tears down built-in fixtures automatically, but contexts you create manually must be closed by you.
9. This model is a key difference from classic WebDriver setups, where isolation usually means starting a new browser session per test - slower and heavier on CI machines.
10. Interview-ready summary: one Browser per worker, one fresh BrowserContext per test for isolation, and one or more Pages per context for tabs and popups.

CODE EXAMPLE

TypeScript test - executed in Chromium (Playwright 1.59.1)

```
import { test, expect } from '@playwright/test';

test('two contexts do not share storage', async ({ browser }) => {
  const userA = await browser.newContext();
  const userB = await browser.newContext();
  const pageA = await userA.newPage();
  const pageB = await userB.newPage();
  await pageA.goto('https://demo.playwright.dev/todomvc/');
  await pageB.goto('https://demo.playwright.dev/todomvc/');

  const input = pageA.getByPlaceholder('What needs to be done?');
  await input.fill('Only in A');
  await input.press('Enter');
  await expect(pageA.getByTestId('todo-title')).toHaveCount(1);

  // Same browser, different context: LocalStorage is not shared
  await pageB.reload();
  await expect(pageB.getByTestId('todo-title')).toHaveCount(0);

  await userA.close();
  await userB.close();
});
```

COMMON MISTAKES

- Sharing one `page` across tests through a global variable or `beforeAll`, which breaks isolation and makes tests order-dependent.
- Creating contexts with `browser.newContext()` and never closing them, leaking memory in long runs.
- Assuming two pages in the same context are isolated - they share cookies and storage.

LIKELY FOLLOW-UPS

- How would you test a chat feature where two users must see each other's messages in real time?
- Which options belong on the context rather than on the page, and why?

Official docs: playwright.dev/docs/browser-contexts | playwright.dev/docs/pages

Q1.04 Which browsers and channels does Playwright support, and how do you run branded Chrome or Edge?

Beginner M1-Q04 | Module 1

ANSWER IN 10 POINTS

1. Playwright supports three engines: Chromium, Firefox and WebKit. Each release pins tested builds of all three and downloads them with `npx playwright install`, so every machine and CI agent runs exactly the same browser versions.
2. In current releases the default Chromium is a Chrome for Testing build, and headless runs use the lighter Chrome Headless Shell. Firefox and WebKit are Playwright-patched builds; WebKit gives Safari-engine coverage on Windows, Linux and macOS.
3. Branded browsers are selected with the `channel` option: `chrome`, `chrome-beta`, `chrome-dev`, `chrome-canary`, `msedge`, `msedge-beta`, `msedge-dev` and `msedge-canary`. Set it per project in `use`, for example `{ ...devices['Desktop Edge'], channel: 'msedge' }`.
4. Branded channels use the browser installed on the machine in its standard location. `npx playwright install chrome` or `npx playwright install msedge` installs the stable branded build, and on a developer laptop that can overwrite the copy you browse with.
5. Typical reason for branded channels: enterprise apps certified on Chrome or Edge, features that differ from open-source Chromium (such as proprietary media codecs), or a compliance checklist that names a specific browser.
6. Mobile coverage comes from device descriptors such as `devices['iPhone 13']` or `devices['Pixel 7']`, which set viewport, user agent, device scale factor and touch. This is emulation on desktop engines, not a real phone.
7. Projects let one config run the same tests across engines; pick a subset with `npx playwright test --project=chromium --project=firefox`. Many teams run Chromium on every pull request and the full matrix nightly to balance feedback time and coverage.
8. On Linux CI use `npx playwright install --with-deps` to install the system libraries the browsers need. `--only-shell` installs just the headless shell for Chromium, and `--no-shell` skips it when you always run headed.
9. Gotcha: an upgrade of `@playwright/test` expects matching browser revisions, so always rerun `npx playwright install` after upgrading, and cache the browser folder in CI keyed on the Playwright version.
10. Interview-ready summary: Chromium, Firefox and WebKit are bundled and version-pinned; branded Chrome or Edge is enabled through the `channel` option per project; mobile is device emulation; and projects plus `-project` control which browsers each pipeline runs.

CODE EXAMPLE

playwright.config.ts - type-checked (strict tsc)

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
    { name: 'webkit', use: { ...devices['Desktop Safari'] } },

    // Branded browsers installed on the machine: npx playwright install chrome msedge
    { name: 'google-chrome', use: { ...devices['Desktop Chrome'], channel: 'chrome' } },
    { name: 'microsoft-edge', use: { ...devices['Desktop Edge'], channel: 'msedge' } },

    // Mobile emulation: viewport, user agent, touch and device scale factor
    { name: 'mobile-safari', use: { ...devices['iPhone 13'] } },
    { name: 'mobile-chrome', use: { ...devices['Pixel 7'] } },
  ],
});
```

COMMON MISTAKES

- Treating the WebKit project as a full substitute for real Safari on macOS or iOS when the business has Safari-specific defects.
- Upgrading `@playwright/test` without rerunning `npx playwright install`, which fails with missing browser executable errors.
- Running every browser and device project on every pull request, which multiplies CI time without a matching gain in defect detection.

LIKELY FOLLOW-UPS

- How would you structure CI so pull requests run fast but you still get full cross-browser coverage?
- What is the difference between running the `chromium` project and running with `channel: 'chrome'`?

Official docs: playwright.dev/docs/browsers | playwright.dev/docs/test-configuration

Q1.05 How do you set up a new Playwright Test project and what does the generated structure contain?

Beginner M1-Q05 | Module 1

ANSWER IN 10 POINTS

1. Run `npm init playwright@latest` in an empty or existing folder with a current LTS version of Node.js. The wizard asks for TypeScript or JavaScript, the tests folder name, whether to add a GitHub Actions workflow, and whether to install browsers now.
2. It adds `@playwright/test` as a dev dependency in `package.json`, creates `playwright.config.ts`, an example spec in the tests folder, optional `.github/workflows/playwright.yml`, and `.gitignore` entries for generated folders such as `test-results/` and `playwright-report/`.
3. Browsers are not stored in the project. They are downloaded to a shared OS cache (for example `%USERPROFILE%\AppData\Local\ms-playwright` on Windows or `~/.cache/ms-playwright` on Linux), so several projects on one machine reuse them.
4. The generated config is a good baseline: `testDir`, `fullyParallel: true`, `forbidOnly` and `retries` driven by `process.env.CI`, the HTML reporter, `trace: 'on-first-retry'`, three browser projects, and a commented-out `webServer` block for starting your app.

5. Everyday commands: `npx playwright test` runs headless across projects, `--headed` shows the browser, `--ui` opens UI Mode with watch and time-travel debugging, `--debug` opens the Inspector, and `npx playwright show-report` opens the last HTML report.
6. For a real project you then add `baseURL` in `use` so tests call `page.goto('/login')`, environment handling through env variables, and folders such as `tests/`, `pages/` (page objects), `fixtures/` and `test-data/`.
7. Authentication is usually added early through a setup project that logs in once and saves `storageState` to a gitignored file, which other projects then reuse instead of logging in through the UI in every test.
8. Install the Playwright Test for VS Code extension: it runs and debugs single tests, shows locators as you hover, and can record new tests directly into a file.
9. Gotcha: keep `@playwright/test` pinned and upgrade intentionally with `npm install -D @playwright/test@latest` followed by `npx playwright install`, because browser revisions are tied to the package version.
10. Interview-ready summary: `npm init playwright@latest` scaffolds the package, config, example tests, optional CI workflow and gitignore entries; you then add `baseURL`, environment handling, page objects, auth setup and a pinned upgrade routine to make it production-ready.

CODE EXAMPLE

Terminal - illustrative, adapt to your project

```
# Scaffold: choose TypeScript, tests folder, optional GitHub Actions workflow, install browsers
npm init playwright@latest

# Typical result
# playwright.config.ts
# package.json           (@playwright/test in devDependencies)
# tests/example.spec.ts
# .github/workflows/playwright.yml   (if selected)
# .gitignore              (test-results/, playwright-report/ ...)
```

```
# Run and inspect
npx playwright test
npx playwright test --project=chromium --headed
npx playwright test --ui
npx playwright show-report
```

```
# Upgrade intentionally, then fetch matching browsers
npm install -D @playwright/test@latest
npx playwright install
```

COMMON MISTAKES

- Committing `test-results/`, `playwright-report/` or saved `storageState` files to git, which bloats the repo and can leak session cookies.
- Hardcoding full environment URLs in every `page.goto()` instead of setting `baseURL` once in the config.
- Installing Playwright globally and relying on a global version, so local and CI runs drift apart.

LIKELY FOLLOW-UPS

- How would you make the same suite run against QA, staging and production-like environments?
- What would you change in the generated config before running it in CI?

Official docs: playwright.dev/docs/intro | playwright.dev/docs/test-configuration

Q1.06 Playwright Library vs Playwright Test: when would you use each?

Intermediate M1-Q06 | Module 1

ANSWER IN 10 POINTS

1. Playwright Library is the automation API itself: you import `chromium`, `firefox` or `webkit`, launch a browser, create contexts and pages, and manage their lifecycle in your own code. Playwright Test is the runner built on top of that API, exposed through `@playwright/test`.
2. With the Library you own everything: launching and closing the browser, creating isolated contexts, error handling, retries, parallelism and reporting. Nothing is automatic, which is flexible but means more code.
3. Playwright Test adds fixtures (`page`, `context`, `browser`, `request`), test isolation per test, parallel workers, sharding, retries, projects, web-first `expect` assertions, HTML and other reporters, and trace, screenshot and video capture configured in one file.
4. Use Playwright Test for any end-to-end, component or API test suite. It is the recommended way to write tests, and the official guides and best practices assume it.
5. Use the Library for automation that is not a test suite: scraping a report, generating PDFs or screenshots, warming caches, synthetic monitoring scripts, or embedding a browser inside a larger Node service or CLI tool.
6. The Library is also what you reach for when another framework owns the test lifecycle, for example integrating with an existing Jest, Mocha or Cucumber setup, where you create and close browsers in that framework's hooks.
7. The APIs are the same objects either way: a `Page` from the Library and the `page` fixture behave identically, so knowledge transfers. The difference is who creates and disposes them.
8. Gotcha: Library scripts must close the browser in a `finally` block, otherwise a thrown error leaves orphaned browser processes on CI agents and developer machines.
9. Another gotcha: mixing both styles, such as calling `chromium.launch()` inside a Playwright Test spec, bypasses the configured projects, `use` options, tracing and fixture cleanup.
10. Interview-ready summary: use Playwright Test for testing because it adds isolation, fixtures, assertions, parallelism and reporting; use the Library for non-test automation or when another framework already owns the lifecycle.

CODE EXAMPLE

TypeScript module - type-checked (strict tsc)

```
import { chromium } from '@playwright/test';

// Library usage: a standalone script, no runner, no fixtures
async function captureDashboard(): Promise<void> {
  const browser = await chromium.launch({ headless: true });
  const context = await browser.newContext({ viewport: { width: 1280, height: 720 } });
  const page = await context.newPage();
  try {
    await page.goto('https://demo.playwright.dev/todomvc/');
    await page.screenshot({ path: 'todomvc.png', fullPage: true });
    console.log('Captured:', await page.title());
  } finally {
    await context.close();
    await browser.close();
  }
}

captureDashboard().catch((error) => {
  console.error(error);
  process.exit(1);
});
```

COMMON MISTAKES

- Building a custom runner on the Library (retries, parallelism, reporting) for a normal test suite, re-implementing what Playwright Test already provides.
- Forgetting `browser.close()` in a `finally` block, leaving zombie browser processes after failures.
- Launching browsers manually inside Playwright Test specs and losing config-driven projects, tracing and automatic cleanup.

LIKELY FOLLOW-UPS

- How would you share login state between a Library-based monitoring script and your Playwright Test suite?
- If your company already uses Cucumber, how would you integrate Playwright and what would you lose compared to Playwright Test?

Official docs: playwright.dev/docs/library | playwright.dev/docs/intro

Q1.07 What is codegen and how should it be used in a real project?

Beginner M1-Q07 | Module 1

ANSWER IN 10 POINTS

1. Codegen is Playwright's test generator. `npx playwright codegen <url>` opens a browser plus the Playwright Inspector; as you click and type, it writes matching Playwright code in real time.
2. It picks locators the way Playwright recommends: role, label, text or test id first, refining until the locator is unique. That makes it a fast way to learn good locators for a complicated page.
3. The Inspector toolbar can add assertions while recording - assert visibility, assert text and assert value - and has a pick-locator mode that shows the best locator for any element you hover.
4. Useful options: `--target` chooses the output language (default `playwright-test`), `-o` writes to a file, `-b` picks the browser, `--device` and `--viewport-size` emulate screens, and `--lang`, `--timezone`, `--geolocation` and `--color-scheme` emulate the environment.
5. For authenticated areas, record once with `--save-storage=auth.json` after logging in, and later start codegen with `--load-storage=auth.json` to land directly in the signed-in app.

6. If your app uses a custom test id attribute, pass `--test-id-attribute=data-qa` so generated locators use it. Inside a test, `await page.pause()` opens the Inspector at that exact state, where you can record further steps.
7. The VS Code extension offers the same features in the editor: record a new test, record at the cursor inside an existing test, and pick a locator to paste into code.
8. In a real project treat output as a draft: replace absolute URLs with `baseURL`, move repeated steps into page objects or fixtures, parameterise test data, delete redundant clicks, and add business-level assertions that codegen cannot infer.
9. Gotcha: recordings capture incidental actions (extra clicks, hover noise) and can miss waits for backend effects, so a recorded script that passes once is not yet a reliable test.
10. Interview-ready summary: codegen is a locator-discovery and scaffolding tool - great for exploring pages, learning locators and bootstrapping flows - but generated code must be refactored into maintainable, assertion-rich tests.

CODE EXAMPLE

Terminal - illustrative, adapt to your project

```
# Record a flow in Chromium; code appears live in the Inspector
npx playwright codegen https://demo.playwright.dev/todomvc/

# Write to a file, record in WebKit with an emulated phone
npx playwright codegen -o tests/todo.spec.ts https://demo.playwright.dev/todomvc/
npx playwright codegen -b webkit --device="iPhone 13" https://demo.playwright.dev/todomvc/

# Log in once and save state, then reuse it for later recordings
npx playwright codegen --save-storage=auth.json https://example.com/login
npx playwright codegen --load-storage=auth.json https://example.com/dashboard

# Custom test id attribute and another output language
npx playwright codegen --test-id-attribute=data-qa https://example.com
npx playwright codegen --target=python-pytest https://example.com
```

COMMON MISTAKES

- Committing raw recordings as the regression suite, ending up with duplicated, assertion-free scripts that are expensive to maintain.
- Committing the `auth.json` file created by `--save-storage`, which contains live session cookies.
- Keeping fragile generated steps such as clicks on layout containers instead of rewriting them to target the meaningful control.

LIKELY FOLLOW-UPS

- Codegen produced `getByText('Submit')` but the test is flaky in another language build. How would you improve that locator?
- How would you use `page.pause()` together with codegen to extend an existing test?

Official docs: playwright.dev/docs/codegen-intro | playwright.dev/docs/codegen

Q1.08 How do you handle new tabs, popups and multiple windows in Playwright?

Intermediate M1-Q08 | Module 1

ANSWER IN 10 POINTS

1. In Playwright every tab or popup window is simply another `Page` object in the same `BrowserContext`. There is no window-handle switching as in Selenium; you hold references to each page and call methods on whichever one you need.
2. When an action opens a new tab, start waiting before the action: `const popupPromise = page.waitForEvent('popup')`, then click, then `await popupPromise`. Waiting after the click creates a race where the event can fire before you listen.
3. `page.waitForEvent('popup')` catches pages opened by that specific page, including `target="_blank"` links and `window.open()`. `context.waitForEvent('page')` catches any new page in the context, which is handy when the opener is unknown.
4. The new page may still be loading when you receive it. Locators and web-first assertions auto-wait, but if you read the URL or title immediately, call `await popup.waitForLoadState()` first.
5. Useful companions: `context.pages()` lists open pages, `popup.opener()` returns the page that opened it, `page.bringToFront()` activates a tab, and `popup.waitForEvent('close')` handles windows that close themselves, such as OAuth sign-in popups.
6. Pages in one context share cookies and storage, so a popup is already signed in as the same user. For a second user, create a separate context instead of a new page.
7. Network mocking set with `page.route()` does not apply to a new popup. Register routes with `context.route()` when the mocked call may happen in a page you have not created yet.
8. Realistic cases: invoice PDFs opening in a new tab, third-party payment or SSO popups, help-centre links, and admin consoles that spawn detail windows. Close popups you open so the next steps do not act on the wrong page.
9. Gotcha: if the click does not actually open a page (blocked by an app bug or a changed `target`), the wait times out. That failure is a genuine signal, so keep the default timeout rather than swallowing the error.
10. Interview-ready summary: start `waitForEvent('popup')` or `context.waitForEvent('page')` before the click, await the resulting `Page`, wait for load if you need URL or title, interact with it directly, and use context-level routes and listeners for pages you do not create.

CODE EXAMPLE

TypeScript test - executed in Chromium (Playwright 1.59.1)

```
import { test, expect } from '@playwright/test';

test('handle a link that opens a new tab', async ({ context, page }) => {
  // context.route also covers the popup page, page.route would not
  await context.route('https://app.example.test/**', (route) =>
    route.fulfill({
      contentType: 'text/html',
      body: route.request().url().endsWith('/help')
        ? '<title>Help</title><h1>Help Centre</h1>'
        : '<a href="/help" target="_blank">Open help</a>',
    })),
  );
  await page.goto('https://app.example.test/');

  const popupPromise = page.waitForEvent('popup'); // Listen before clicking
  await page.getByRole('link', { name: 'Open help' }).click();
  const helpTab = await popupPromise;
  await helpTab.waitForLoadState();

  await expect(helpTab).toHaveTitle('Help');
  await expect(helpTab.getByRole('heading', { name: 'Help Centre' })).toBeVisible();
  expect(context.pages()).toHaveLength(2);

  await helpTab.close();
  await expect(page.getByRole('link', { name: 'Open help' })).toBeVisible();
});
```

COMMON MISTAKES

- Calling `waitForEvent('popup')` after the click instead of creating the promise before it, causing intermittent timeouts.
- Looking for Selenium-style window switching APIs instead of simply holding a reference to the new `Page`.
- Mocking APIs with `page.route()` and wondering why the popup still hits the real backend.

LIKELY FOLLOW-UPS

- How would you test a Google or Microsoft SSO login that happens in a popup window?
- A click sometimes opens a new tab and sometimes navigates the same tab. How would you write a robust test?

Official docs: playwright.dev/docs/pages | playwright.dev/docs/api/class-page

Q1.09 How do you work with iframes in Playwright?

Intermediate M1-Q09 | Module 1

ANSWER IN 10 POINTS

1. Elements inside an iframe belong to a different document, so page-level locators cannot see them. You enter the frame first and then locate inside it.
2. `page.frameLocator('iframe[title="Payment"]')` returns a `FrameLocator`; chain any locator on it, such as `.getByLabel('Card number')`. Like regular locators it is lazy, auto-waits for the iframe to attach and load, and is strict when several frames match.
3. If you already have a `Locator` pointing to the iframe element, `locator.contentFrame()` converts it to a `FrameLocator`, and `frameLocator.owner()` goes back to the iframe element. This keeps one consistent locator style for frames and elements.

4. Nested iframes are handled by chaining:
`page.frameLocator('#outer').frameLocator('#inner').getByRole('button', { name: 'Pay' })`. There is no switch-to and switch-back step as in Selenium.
5. The lower-level `Frame` API is also available: `page.frame({ name })` or `page.frame({ url })` returns a `Frame` or `null`, `page.frames()` lists all frames, and `page.mainFrame()` is the top document. Use it for frame-level calls such as `frame.url()` or `frame.evaluate()`.
6. Cross-origin iframes (payment gateways, embedded maps, chat widgets, captcha providers) work through the same API because Playwright attaches to every frame regardless of origin.
7. Prefer a stable attribute on the iframe element itself - `title`, `name`, or a test id - over its position or a generated `src` URL that changes per session or environment.
8. Assertions work inside frames exactly as on the page: `await expect(frame.getByText('Payment successful')).toBeVisible()` retries until the frame content updates.
9. Gotcha: when an app replaces an iframe (for example re-rendering a payment widget), a `Frame` object you stored can become detached, while a `FrameLocator` simply re-resolves on the next action. Prefer frame locators in tests.
10. Interview-ready summary: use `page.frameLocator()` or `locator.contentFrame()` to scope locators into an iframe, chain them for nested frames, rely on auto-waiting and strictness as with normal locators, and keep the `Frame` API for frame-level operations.

CODE EXAMPLE

TypeScript test - executed in Chromium (Playwright 1.59.1)

```
import { test, expect } from '@playwright/test';

test('fill a field inside an iframe', async ({ page }) => {
  await page.setContent(
    '<h1>Checkout</h1>' +
    '<iframe name="payment" title="Payment" ' +
    'srcdoc="<label>Card number <input></label><button>Pay</button>"></iframe>',
  );

  const payment = page.frameLocator('iframe[title="Payment"]');
  await payment.getByLabel('Card number').fill('4111 1111 1111 1111');
  await expect(payment.getByLabel('Card number')).toHaveValue('4111 1111 1111 1111');

  // Same frame reached from a Locator to the iframe element
  const sameFrame = page.locator('iframe[name="payment"]').contentFrame();
  await expect(sameFrame.getByRole('button', { name: 'Pay' })).toBeEnabled();

  // Page-Level Locators do not look inside the iframe document
  await expect(page.getByLabel('Card number')).toHaveCount(0);

  // Frame API for frame-level operations
  const frame = page.frame({ name: 'payment' });
  expect(frame).not.toBeNull();
});
```

COMMON MISTAKES

- Using page-level locators for elements inside an iframe and concluding the element is 'not found' or 'not visible'.
- Locating iframes by index or by a dynamic `src` query string that changes per session.
- Storing a `Frame` object across re-renders instead of using a `FrameLocator` that re-resolves.

LIKELY FOLLOW-UPS

- How would you assert something inside an iframe that is nested three levels deep and loads lazily?
- What is the difference between `page.frameLocator()` and `page.frame()`, and when would you choose each?

Official docs: playwright.dev/docs/frames | playwright.dev/docs/api/class-frame-locator

Q1.10 How do you handle JavaScript dialogs, file uploads and downloads in Playwright?

Intermediate M1-Q10 | Module 1

ANSWER IN 10 POINTS

1. By default Playwright auto-dismisses `alert`, `confirm` and `prompt` dialogs, so a test does not hang. To assert on or accept a dialog, register a handler with `page.on('dialog')` or `page.once('dialog')` before the action that triggers it.
2. Inside the handler use `dialog.type()`, `dialog.message()` and `dialog.defaultValue()` to inspect it, then call `dialog.accept()` (with optional prompt text) or `dialog.dismiss()`. A registered handler must do one of these, otherwise the page stays blocked and the triggering action stalls.
3. `beforeunload` dialogs only appear when closing with `page.close({ runBeforeUnload: true })`; handle them with the same dialog listener when testing 'unsaved changes' warnings.
4. For uploads, call `locator.setInputFiles()` on an `<input type="file">` with a path, an array of paths, or an in-memory payload `{ name, mimeType, buffer }`. Passing an empty array clears the selection.
5. When the page uses a custom button that opens the native picker without exposing an input, start `page.waitForEvent('filechooser')` before clicking, then call `fileChooser.setFiles()` on the result.
6. For downloads, start `page.waitForEvent('download')` before the click, await the `Download`, then use `download.suggestedFilename()`, `download.saveAs(path)` to keep it, or `download.path()` to read the temporary file.
7. Downloaded files are temporary and deleted when the browser context closes, so save or read them inside the test. Verify content such as CSV headers or row counts, not just that a file arrived.
8. In-memory upload payloads keep tests self-contained and CI-friendly; for large fixtures, store files under a test-data folder and build the path with `path.join(__dirname, ...)` rather than relative to the current working directory.
9. Gotcha: registering a dialog handler after the triggering click is too late, and a permanent `page.on('dialog')` handler that always accepts can hide unexpected confirmations; prefer `page.once` scoped to the step under test.
10. Interview-ready summary: register dialog, filechooser or download waits before the triggering action; accept or dismiss dialogs explicitly; upload with `setInputFiles` (paths or buffers); and validate downloads through `suggestedFilename`, `saveAs` or `path` before the context closes.

CODE EXAMPLE

TypeScript test - executed in Chromium (Playwright 1.59.1)

```
import { test, expect } from '@playwright/test';
import { readFileSync } from 'node:fs';

test('confirm dialog, upload and download', async ({ page }) => {
  await page.setContent(
    '<button id="del">Delete</button><p id="out"></p>' +
    '<script>document.getElementById("del").onclick = () => {' +
    ' document.getElementById("out").textContent = confirm("Delete record?") ? "Deleted" : "Kept"; }' +
    '</script>' +
    '<input type="file" aria-label="Resume">' +
    '<a download="report.csv" href="data:text/csv,id,name%0A1,Ravi">Export CSV</a>',
  );

  page.once('dialog', async (dialog) => {
    expect(dialog.type()).toBe('confirm');
    expect(dialog.message()).toBe('Delete record?');
    await dialog.accept();
  });

  await page.getByRole('button', { name: 'Delete' }).click();
  await expect(page.locator('#out')).toHaveText('Deleted');

  await page.getByLabel('Resume').setInputFiles({
    name: 'resume.txt', mimeType: 'text/plain', buffer: Buffer.from('QA Engineer'),
  });
  await expect(page.getByLabel('Resume')).toHaveValue(/resume/);

  const downloadPromise = page.waitForEvent('download');
  await page.getByRole('link', { name: 'Export CSV' }).click();
  const download = await downloadPromise;
  expect(download.suggestedFilename()).toBe('report.csv');
  expect(readFileSync(await download.path(), 'utf8')).toContain('Ravi');
});
```

COMMON MISTAKES

- Registering the dialog listener after the click, or trying to locate the native dialog with a DOM locator.
- Clicking an upload button and expecting to interact with the operating system file picker instead of using `setInputFiles` or the `filechooser` event.
- Only checking that a download event fired without verifying the file name or content, or reading the file after the context has closed.

LIKELY FOLLOW-UPS

- How would you test a drag-and-drop upload area that has no visible file input?
- How would you validate the contents of a downloaded Excel or PDF report in CI?

Official docs: playwright.dev/docs/dialogs | playwright.dev/docs/input | playwright.dev/docs/downloads

Playwright Interview Cheat Sheet

Playwright Interview Mastery by Ravi - commands, APIs and answer templates for @playwright/test 1.59 (TypeScript)

CLI essentials

Run all / one file:line - Omit the path to run everything in testDir

```
npx playwright test tests/cart.spec.ts:42
```

UI mode - Watch mode, time-travel DOM snapshots, re-run single tests

```
npx playwright test --ui
```

Debug with Inspector - Headed, workers=1, timeout=0; step through each action

```
npx playwright test --debug
```

Headed, one project

```
npx playwright test --headed --project=chromium
```

Filter by tag or title

```
npx playwright test --grep @smoke --grep-invert @slow
```

Repeat to expose flakes - `--last-failed` re-runs only the failures from the previous run

```
npx playwright test cart.spec.ts --repeat-each=20
```

Open HTML report - A trace alone: `npx playwright show-trace path/to/trace.zip`

```
npx playwright show-report
```

Record a test - Generates role-based locators; treat output as a draft

```
npx playwright codegen demo.playwright.dev/todomvc
```

Browsers + OS deps

```
npx playwright install --with-deps chromium
```

Locator priority (best first)

Role + accessible name - Matches how users and screen readers see the page

```
page.getByRole('button', { name: 'Sign in' })
```

Form label

```
page.getByLabel('Email address')
```

Placeholder / alt text - Also `getByAltText('Company logo')` for images, `getTitle()`

```
page.getByPlaceholder('Search products')
```

Visible text - Default is substring, case-insensitive; exact: true for full match

```
page.getByText('Welcome back', { exact: true })
```

Test id - Uses `data-testid`; change with `use.testIdAttribute` in config

```
page.getByTestId('cart-count')
```

CSS / XPath (last resort) - Coupled to DOM structure; breaks on markup refactors

```
page.locator('#legacy-grid tr.selected')
```

Locator refinement

Filter by text

```
page.getByRole('row').filter({ hasText: 'ORD-1042' })
```

Filter by child

```
rows.filter({ has: page.getByRole('button', { name: 'Edit' }) })
```

Exclude

```
rows.filter({ hasNotText: 'Archived' })
```

Chain (scope)

```
page.getByRole('dialog').getByRole('button', { name: 'OK' })
```

Position - nth is zero-based; use only when order is the behaviour under test

```
items.first(); items.last(); items.nth(2)
```

Both / either - `or` may match 2 elements: add `.first()` to avoid a strict-mode error

```
saveBtn.and(page.getByTitle('Save draft'));
newBtn.or(dialog)
```

iframe - Also: `page.locator('#payment').contentFrame()`

```
page.frameLocator('#payment').getByLabel('Card number')
```

Strictness - An action on a locator matching 2+ elements throws a strict mode violation

Actions

Navigate - Relative URLs resolve against `use.baseURL`

```
await page.goto('/login');
```

Fill / clear - fill replaces the value; use `clear()` to empty a field

```
await page.getByLabel('Email').fill('qa@example.com');
```

Type key by key - For autocomplete inputs; `page.type` is deprecated

```
await search.pressSequentially('play', { delay: 50 });
```

Check / select

```
await terms.check(); await country.selectOption('IN');
```

Upload file

```
await
page.getByLabel('Resume').setInputFiles('data/cv.pdf');
```

Keys / hover / drag

```
await input.press('Enter'); await card.dragTo(doneColumn);
```

New tab (popup) - Start waiting, click the link, then `await popupPromise`

```
const popupPromise = page.waitForEvent('popup');
```

Dialogs - Without a listener, alert/confirm dialogs are auto-dismissed

```
page.once('dialog', (d) => d.accept());
```

Web-first assertions (auto-retry)

Visible / hidden

```
await expect(toast).toBeVisible(); await
expect(spinner).toBeHidden();
```

Text - toContainText for substring; arrays match lists of elements

```
await expect(total).toHaveText(/Total: \d+/);
```

Page URL / title

```
await expect(page).toHaveURL(/\/dashboard/);
```

Value / state - Also toBeEnabled, toBeDisabled, toBeChecked, toBeFocused

```
await expect(email).toHaveValue('qa@example.com');
```

Attribute / count - toHaveAttribute('aria-selected', 'true') for state flags

```
await expect(items).toHaveCount(3);
```

Soft assertion - Records the failure and lets the test continue

```
await expect.soft(price).toHaveText('499');
```

Poll any value - Retries a function (API, DB) until the matcher passes

```
await expect.poll(() => getStatus(id)).toBe('DONE');
```

Anti-pattern - isVisible() checks once, no retry - flaky. Use toBeVisible()

```
expect(await toast.isVisible()).toBe(true);
```

Timeouts and waiting

Test timeout (30s default) - Config: timeout: 30_000. test.slow() triples it

```
test.setTimeout(60_000);
```

Expect timeout (5s default) - Config: expect: { timeout: 10_000 }

```
await expect(row).toBeVisible({ timeout: 10_000 });
```

Action / navigation - Default 0 = no own limit; bounded by the test timeout

```
use: { actionTimeout: 10_000, navigationTimeout: 30_000 }
```

Wait for URL

```
await page.waitForURL('**/dashboard');
```

Wait for a response

```
const res = await page.waitForResponse('**/api/orders');
```

Load state - 'networkidle' is discouraged; assert on UI state instead

```
await page.waitForLoadState('domcontentloaded');
```

Never in final code - Hard sleep: slow and still flaky. Debugging only

```
await page.waitForTimeout(3000);
```

Config, projects, parallelism

Run shape - Wrap config in defineConfig({...}) from '@playwright/test'

```
fullyParallel: true, forbidOnly: !!process.env.CI,
```

Retries / workers - Omit workers to default to 50% of logical CPU cores

```
retries: process.env.CI ? 2 : 0, workers: process.env.CI ? 4 : 2,
```

Shared options

```
use: { baseURL: 'http://localhost:3000', trace: 'on-first-retry' },
```

Reporters

```
reporter: [['html', { open: 'never' }], ['list']],
```

Start the app - Add reuseExistingServer: !process.env.CI for local runs

```
webServer: { command: 'npm start', url: 'http://localhost:3000' },
```

Browser / device project

```
{ name: 'mobile', use: { ...devices['Pixel 7'] } }
```

Serial file (rarely) - Default: files in parallel, tests in a file in order in one worker

```
test.describe.configure({ mode: 'serial' });
```

Tags

```
test('checkout', { tag: '@smoke' }, async ({ page }) => {});
```

Fixtures

Built-ins - Test-scoped: page, context, request. Worker-scoped: browser, browserName

Declare

```
export const test = base.extend<{ todo: TodoPage }>({ /* ... */ });
```

Fixture body

```
todo: async ({ page }, use) => { await use(new TodoPage(page)); },
```

Teardown - Code after await use(value) runs as teardown, even if the test failed

Worker scope - Created once per worker process - good for expensive setup

```
db: [async ({}, use) => use(await connect()), { scope: 'worker' }],
```

Auto fixture - Runs for every test without being requested

```
logs: [async ({ page }, use) => { await use(); }, { auto: true }],
```

Option fixture - Override per project/file: test.use({ env: 'prod' })

```
env: ['staging', { option: true }],
```

Combine files

```
export const test = mergeTests(authTest, apiTest);
```

Auth and storageState

Setup project

```
{ name: 'setup', testMatch: /\.*\setup\.ts/ },
```

Depend on it - Guarantees the state file exists before e2e tests start

```
{ name: 'e2e', dependencies: ['setup'] },
```

Save after login

```
await page.context().storageState({ path: authFile });
```

Reuse - Gitignore playwright/.auth - it holds live session tokens

```
use: { storageState: 'playwright/.auth/user.json' },
```

Logged-out test

```
test.use({ storageState: { cookies: [], origins: [] } });
```

Login via API - Then request.storageState({ path: authFile }); faster than UI

```
await request.post('/api/login', { data: { user, pass } });
```

Two roles, one test

```
const admin = await browser.newContext({ storageState: adminFile });
```

What is saved - Cookies + localStorage per origin; IndexedDB with indexedDB: true; not sessionStorage

Network mocking and API

Mock a response - Register before goto; context.route() also covers popups

```
await page.route('**/api/products', (r) => r.fulfill({
  json: data }));
```

Block resources

```
await page.route('**/*.png,jpg,svg', (r) => r.abort());
```

Patch real response

```
const res = await route.fetch(); const json = await
res.json();
```

...then fulfill

```
await route.fulfill({ response: res, json: { ...json,
stock: 0 } });
```

Wait for a call - Create before the click, then await resP - no race

```
const resP = page.waitForResponse('**/api/orders');
```

API test

```
const res = await request.post('/api/todos', { data: todo
});
```

Assert 2xx - Then expect(await res.json()).toMatchObject(todo)

```
await expect(res).toBeOK();
```

Replay a HAR - Record once with update: true; route only matching URLs

```
await page.routeFromHAR('hars/api.har', { url: '**/api/**
});
```

Debugging and artifacts

Trace on retry (CI) - 'retain-on-failure' locally; open with show-trace or trace.playwright.dev

```
use: { trace: 'on-first-retry' }
```

Screenshots / video

```
use: { screenshot: 'only-on-failure', video: 'retain-on-
failure' }
```

Pause - Opens the Inspector in headed mode; never commit it

```
await page.pause();
```

Readable steps

```
await test.step('Add item to cart', async () => { /* ...
*/ });
```

Attach evidence

```
await testInfo.attach('log', { body: text, contentType:
'text/plain' });
```

Browser console

```
page.on('console', (msg) => console.log(msg.type(),
msg.text()));
```

Verbose API log - PowerShell: \$env:DEBUG='pw:api'; npx playwright test

```
DEBUG=pw:api npx playwright test
```

CI and Docker

Official image - Tag must match the installed @playwright/test version

```
mcr.microsoft.com/playwright:v1.59.1-noble
```

Run container - --ipc=host avoids Chromium crashing on low shared memory

```
docker run -it --rm --ipc=host
mcr.microsoft.com/playwright:v1.59.1-noble
```

CI install

```
npm ci && npx playwright install --with-deps
```

Shard + blob - Each shard writes ./blob-report; upload it as an artifact

```
npx playwright test --shard=1/4 --reporter=blob
```

Merge reports

```
npx playwright merge-reports --reporter html ./all-blob-
reports
```

PR fast lane

```
npx playwright test --only-changed=origin/main
```

Artifacts - Upload playwright-report/ and test-results/ even when the job fails

Flakiness checklist

No hard sleeps - Assert the state the user waits for instead of waitForTimeout

```
await expect(toast).toBeVisible();
```

Unique test data

```
const sku = 'SKU-' + testInfo.workerIndex + '-' +
Date.now();
```

Freeze time

```
await page.clock.setFixedTime(new Date('2026-01-
15T10:00:00'));
```

Await the data - Wait for the API response before asserting rendered rows

Isolate - No test depends on another's state; avoid serial mode and shared users

Mock third parties - Route payments, maps, analytics; keep your own backend real in E2E

Retries are a signal - Fails the run if any test passed only on retry

```
npx playwright test --retries=2 --fail-on-flaky-tests
```

30-second answer templates

Explain auto-waiting - Actions wait until the target is visible, stable, enabled, unobscured; expect() retries: no sleeps.

Locator strategy - User-facing first: getByRole, getByLabel, getByText; getByTestId when ambiguous; CSS/XPath last.

Why Playwright? - Auto-wait, isolated contexts, built-in runner, traces, network mocking, one API for 3 engines.

Login handling - A setup project logs in once and saves storageState; test projects depend on it and reuse it.

POM vs fixtures - Page objects hold locators and flows; fixtures build and inject them, so tests stay short.

Flaky test - Reproduce with --repeat-each, read the trace, fix the root cause; retries are a signal, not a fix.

A print-ready 2-page version of this cheat sheet is supplied as Cheat-Sheet.pdf.

FULL PACK

What is in the full pack

This free sample contains Module 1 and the cheat sheet. All 80 answers can also be read free online at <https://raviacn95.github.io/> (Playwright category). The full Playwright Interview Mastery pack (₹2,000) is the offline Word + PDF edition and adds:

- **80 core questions, fully answered** - 10-point answers, a TypeScript snippet for every technical question, common mistakes, likely follow-ups and a level tag. Free to read on LearnHub; the pack gives you the formatted offline Word + PDF edition.
- **400+ practice questions** - An index of additional original questions grouped by module for self-testing and mock rounds.
- **2-page cheat sheet** - CLI, locators, assertions, fixtures, auth, network, debugging and CI on two dense A4 pages.
- **20-question timed mock test** - 30 minutes, 14 MCQ + 6 short answers, with an answer key, explanations and a scoring guide.
- **2 runnable sample projects** - todo-app-pom (POM + fixtures on demo.playwright.dev/todomvc) and api-ui-hybrid (local API + UI, auth setup project, network mocking).
- **Checked against Playwright 1.59** - Code snippets type-checked; a representative set executed with @playwright/test 1.59. Delivered as Word (.docx) + PDF.

#	Module	Outcome
1	Playwright Basics & Architecture	Explain browsers, contexts, pages and how Playwright differs from WebDriver tools.
2	Locators Mastery	Choose resilient role-based locators and refine them with filter, chaining and frames.
3	Auto-waiting & Assertions	Describe actionability checks and use web-first, soft and polling assertions correctly.
4	Test Runner, Fixtures & Configuration	Design custom fixtures, projects, timeouts and parallel runs with confidence.
5	Authentication & storageState	Set up login once with a setup project and handle multiple roles safely.
6	Network Mocking & API Testing	Mock, patch and wait on network calls and write API tests with the request fixture.
7	Debugging, Trace Viewer & Flakiness	Debug failures from traces and explain a systematic approach to flaky tests.
8	CI/CD, Scaling & Framework Design	Discuss Docker, sharding, merged reports and how you would structure a framework.

Get the full pack: ₹2,000

Checkout link coming soon. Details and updates: <https://raviacn95.github.io/> (Playwright category).

Not useful for you? Email within 7 days of purchase for a full refund.

Not affiliated with Microsoft or the Playwright project. Playwright is an open-source project by Microsoft. Questions are original and reflect topics commonly tested in QA/SDET interviews at service and product companies; they are not leaked or official questions from

any company. No job or interview outcome is guaranteed.

(c) 2026 Ravi. Free sample - you are welcome to share this PDF unmodified.